



Hashing Algoritmaları
2025-2026 Güz Yarıyılı
Algoritma Analizi
Ödev – 1

Öğrenci Adı: Batuhan Odçıkın

Öğrenci Numarası: 22011093

Dersin Öğretmeni: M. Elif Karslıgil

Video Linki: <https://youtu.be/4MDbV1ifyyU>

1- Problemin Çözümü:

Bu çalışmada, verilen büyük ölçekli veri kümesi üzerinde hash tablosu kullanılarak ekleme ve arama işlemleri gerçekleştirilmiştir. Program başlangıcında kullanıcıdan maksimum probing sayısı (k) alınmaktadır. Bu değer, hash tablosunda çakışma (collision) durumunda lineer probing yöntemiyle kaç deneme yapılacağını belirlemektedir.

Veri kümesi, ilk satırında eleman sayısı olmak üzere bir .txt dosyasından okunmaktadır. Okunan veri sayısı x olarak belirlenmiş ve bu değere göre dinamik olarak bir tamsayı dizisi oluşturulmuştur. Hash tablosunun boyutu, çakışmaları azaltmak amacıyla $x / 10$ değerinden büyük veya eşit en yakın asal sayı olarak seçilmiştir.

Verilerin hash tablosuna eklenmesi sırasında, öncelikle lineer probing yöntemi ile k kez boş yer aranmıştır. Eğer bu denemeler sonucunda uygun bir boş hücre bulunamazsa, ilgili hash indeksinde chaining (bağlı liste) yöntemi kullanılarak eleman kuyruğa eklenmiştir. Bu yaklaşım, lineer probing ve chaining yöntemlerinin hibrit bir şekilde kullanılmasını sağlamaktadır.

Arama işlemi sırasında da benzer bir yaklaşım izlenmiştir. Öncelikle k deneme boyunca lineer probing uygulanmış, aranan eleman bulunamazsa ilgili indeksin bağlı listesi üzerinde arama yapılmıştır. Her eleman için kaç denemede bulunduğu bilgisi `try_count_array` dizisine kaydedilmiştir.

Son aşamada, her bir eleman için yapılan probing sayıları analiz edilerek maksimum probing ve ortalama probing değerleri hesaplanmış ve kullanıcıya sunulmuştur.

2- Karşılaşılan Sorunlar:

İlk olarak linear probing uygulamak için hash tablosunu integer olarak tanımlamıştım fakat sonrasında chaining yapmaya uygun olmadığı için Node yapısına geçmem gerekti. Data ve Next değişkenlerini içeren bir Node structında sakladım. Sonrasında bu pointerı fonksiyon içerisinde pointer olarak verirken tek pointer olarak verdiğim için `calloc` ile doğru yerde memory allocate edemiyordum. Bu problemi de çift pointer parametre olarak çözdüm. Hash map tanımlarken de next pointerını ilk başta NULL olarak set etmiyordum o sebeple bulamazsa sonsuza kadar arama riski vardı bu sebeple hash table'ın elemanlarının her birinin next'ini NULL olarak initialize ederek daha güvenli bir veri yapısı oluşturmuş oldum. Son olarak da h fonksiyonu ile elde ettiğim indexi linear probing ile $index + ctr$ yaptıktan sonra tekrar m ile modunu almadığım için eğer m'e yakın bir index çıkarsa ve k değeri ile indexin toplamı m'i aşarsa allocate edilmeyen bir yere veri depolanmış oluyor ve hem kaybolmuş oluyor hem de hash table'ı bozmuş oluyordu. Elde ettiğim $index + ctr$ değerini de $mod\ m$ yaparak tekrar listenin başının taranmasını sağlamış oldum.

3- Karmaşıklık Analizi:

Hash Table Insertion

Her bir eleman için:

- En fazla k adet lineer probing denemesi yapılır.
- Probing başarısız olursa bağlı listeye ekleme yapılır.

Bu durumda ekleme işleminin ortalama karmaşıklığı: $O(k)$

k sabit kabul edildiğinde: $O(1)$

Hash Table Search

Arama işlemi iki aşamadan oluşmaktadır:

1. Lineer probing (en fazla k adım)
2. Chaining (bağlı liste üzerinde arama)

En kötü durumda arama karmaşıklığı: $O(k + L)$

Burada L , ilgili hash indeksindeki bağlı listenin uzunluğudur.

Ortalama durumda, uygun hash boyutu ve asal sayı seçimi sayesinde: $O(1)$

4- Ekran Çıktıları:

ID-1_5M.txt ödevde verilen, random_number.txt 1.1 Milyonluk custom oluşturulan numara listesidir.

```
Enter the probing count (k):1
Probing count = 1
File Path: 'ID-1_5M.txt'
Maximum probing = 9
Average probing = 4
```

```
Enter the probing count (k):1
Probing count = 1
File Path: 'random_numbers.txt'
Maximum probing = 9
Average probing = 4
```

```
Enter the probing count (k):5
Probing count = 5
File Path: 'ID-1_5M.txt'
Maximum probing = 14
Average probing = 8
```

```
Enter the probing count (k):5
Probing count = 5
File Path: 'random_numbers.txt'
Maximum probing = 14
Average probing = 8
```

```
Enter the probing count (k):10
Probing count = 10
File Path: 'ID-1_5M.txt'
Maximum probing = 19
Average probing = 12
```

```
Enter the probing count (k):10
Probing count = 10
File Path: 'random_numbers.txt'
Maximum probing = 19
Average probing = 12
```

```
Enter the probing count (k):25
Probing count = 25
File Path: 'ID-1_5M.txt'
Maximum probing = 34
Average probing = 26
```

```
Enter the probing count (k):25
Probing count = 25
File Path: 'random_numbers.txt'
Maximum probing = 34
Average probing = 26
```

```
Enter the probing count (k):100
Probing count = 100
File Path: 'ID-1_5M.txt'
Maximum probing = 109
Average probing = 94
```

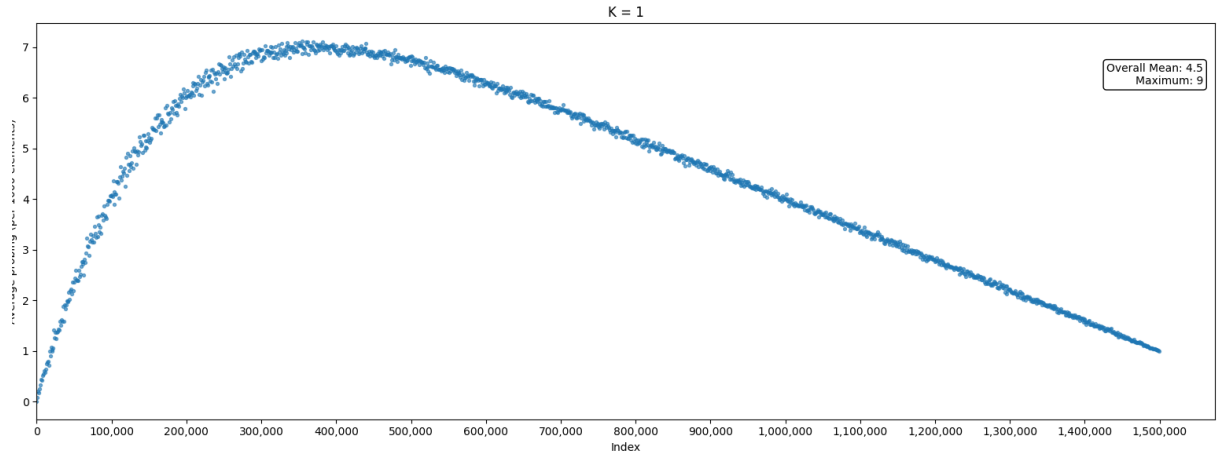
```
Enter the probing count (k):100
Probing count = 100
File Path: 'random_numbers.txt'
Maximum probing = 109
Average probing = 93
```

```
Enter the probing count (k):500
Probing count = 500
File Path: 'ID-1_5M.txt'
Maximum probing = 509
Average probing = 455
```

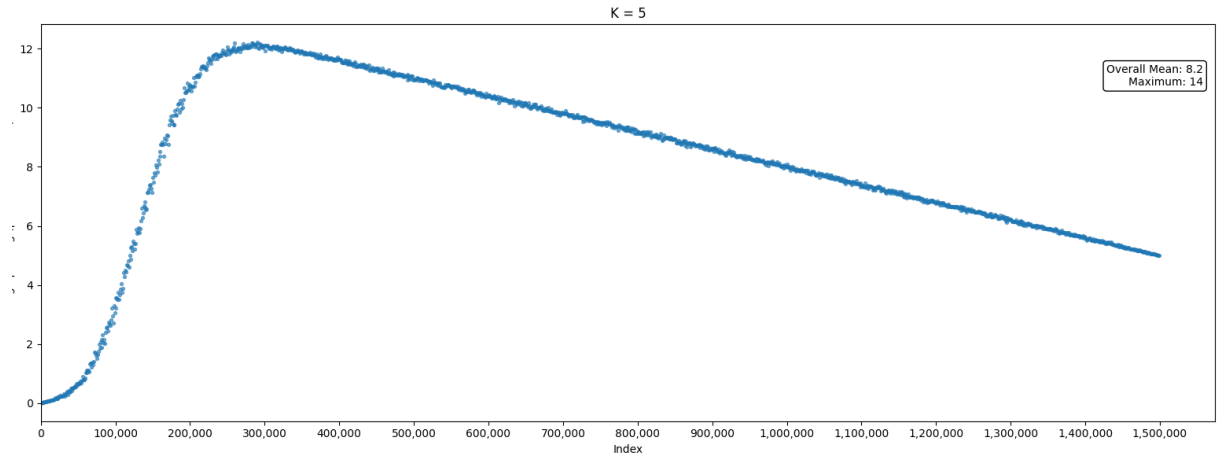
```
Enter the probing count (k):500
Probing count = 500
File Path: 'random_numbers.txt'
Maximum probing = 509
Average probing = 454
```

```
Enter the probing count (k):1000
Probing count = 1000
File Path: 'ID-1_5M.txt'
Maximum probing = 1009
Average probing = 905
```

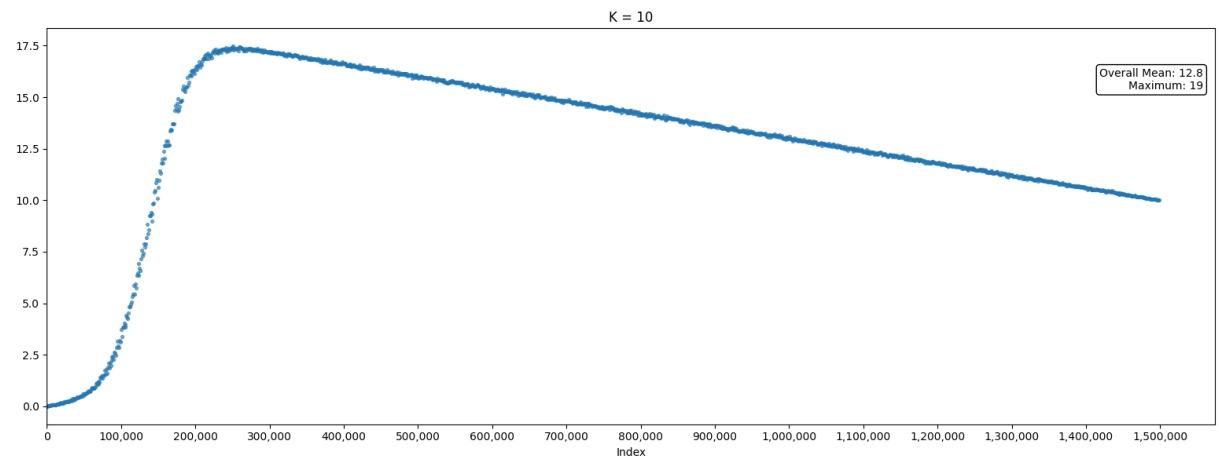
```
Enter the probing count (k):1000
Probing count = 1000
File Path: 'random_numbers.txt'
Maximum probing = 1009
Average probing = 905
```

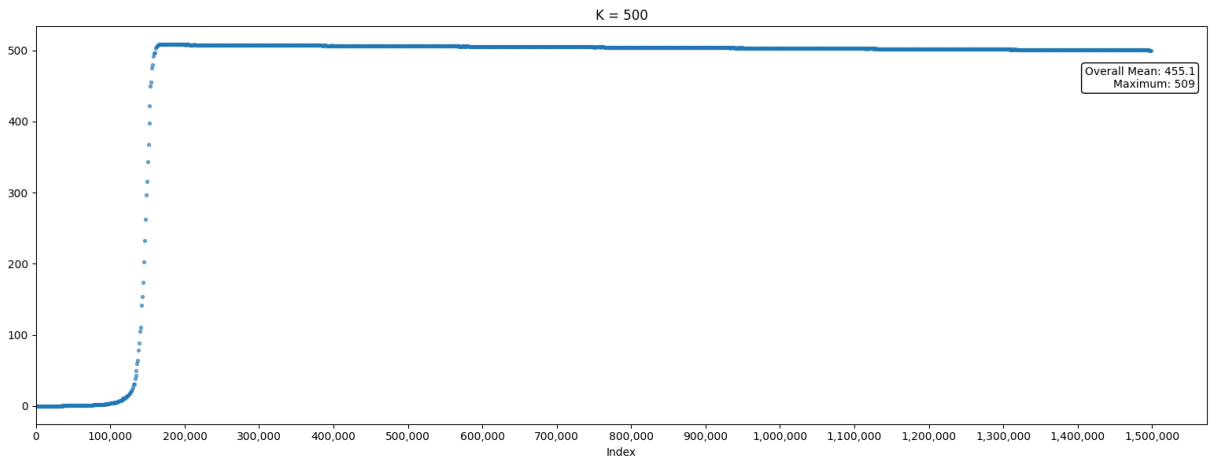
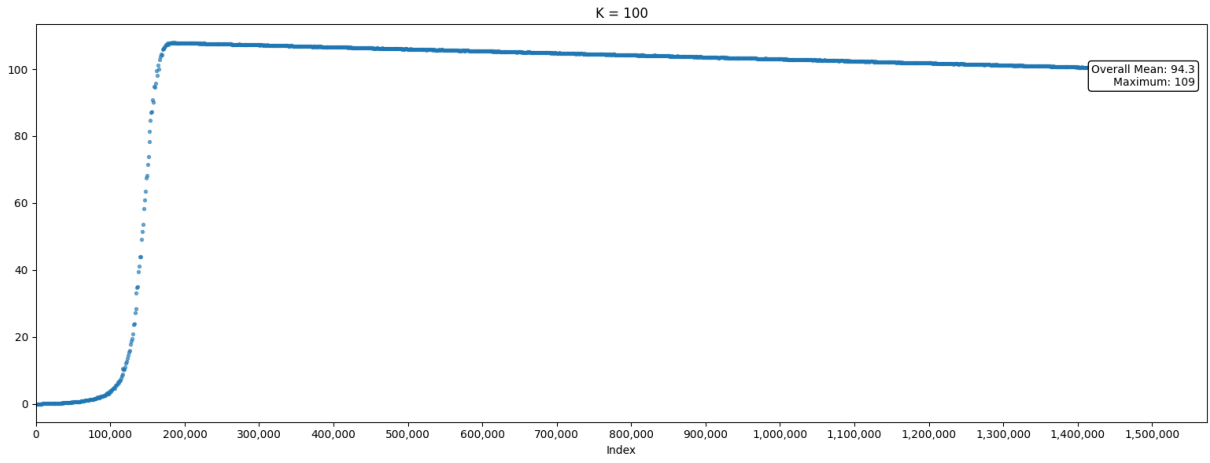
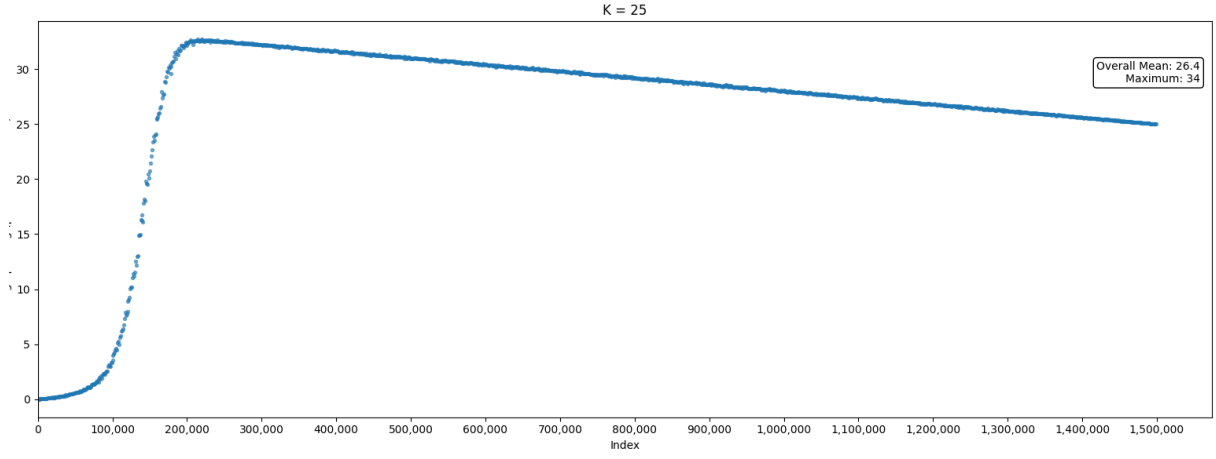


K = 1 olduğu zaman ilk denemelerde daha sıklıkla linkli liste kullanılmakta fakat bu sayede farklı katanlar için de yer kalmaktadır. Bu sayede arama hızlı şekilde yapılabilmektedir, hızlıca linkli liste ramasına geçilmektedir.

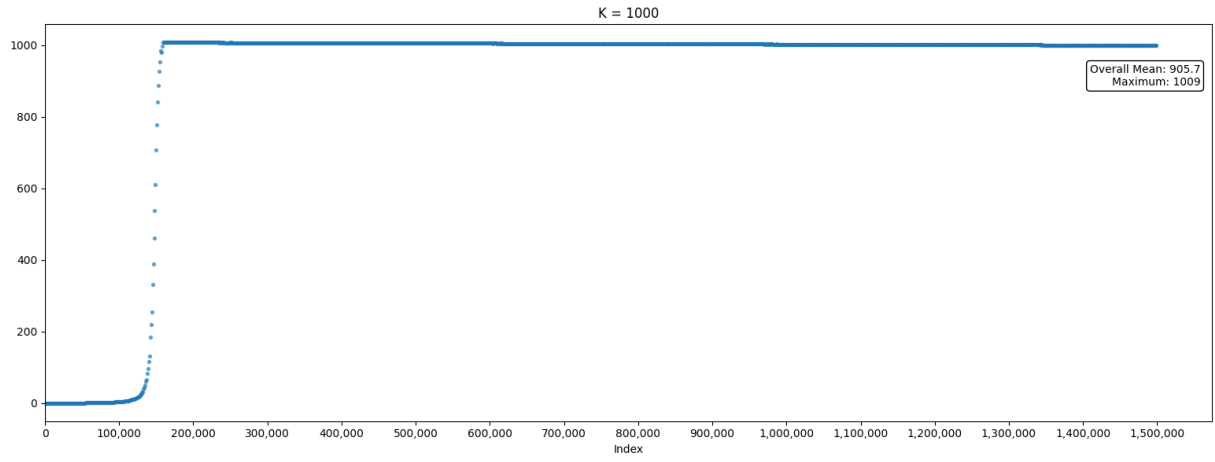


K arttıkça linear search ile vakit kaybı artmaktadır.





K = 500 olduğunda artık neredeyse her eleman için teker teker lineer arama yapılmaktadır. Bu da linkli listedeki eleman için +500 lük bir arama getirmektedir.



Linkli listeye artık 1000 arama itibari ile bakılmaya başlandığı için ilk elemanlar dışında tüm elemanlar için teker teker 1000 elemana bakılmaktadır.